

“카드 바꾸기” 문제 풀이

작성자: 박선재

부분문제 1

N 이 3 이하이다. N 이 2 이상이므로, $N = 2$ 일 때와 $N = 3$ 일 때로 나누어 생각할 수 있다.

우선 $N = 2$ 이면 바꿀 필요가 없다. 따라서 $N = 2$ 일 경우 답은 0이다. $N = 3$ 일 경우, 카드에 적혀있는 수가 왼쪽부터 순서대로 a, b, c 라고 했을 때, $c - b = b - a$ 이면, 바꿀 필요가 없다. 이때 답은 0이다. $c - b \neq b - a$ 이면, $d = b - a$ 라고 했을 때, 마지막 수를 $b + d$ 로 바꾸면 되므로, 답이 1이 된다. 따라서 경우를 나누어 문제를 해결할 수 있다. 시간 복잡도는 $O(1)$ 이다. 제한 시간 내에 충분히 해결할 수 있다.

부분문제 2

먼저, 수들을 적절히 바꾼 후 최종상태에서 카드에 적혀있는 수들은 적절한 정수 a, d 에 대해 왼쪽부터 순서대로 $a, a + d, \dots, a + (N - 1)d$ 가 돼야 한다.

$N \leq 5$ 인 경우, 원래 카드 중 2개 이상의 카드에 적혀 있는 수들은 바뀌지 않으므로, 선택할 수 있는 모든 카드 쌍에 대해 그 카드들에 적혀있는 수를 바꾸지 않을 수 있는지 판단한다. 가능하다면, 위의 a, d 가 정해지므로, 적혀있는 수를 바꿔야 하는 카드의 개수를 세어준다. 그 후, 각 경우에서 적혀있는 수를 바꿔야 하는 카드 개수 중 최솟값을 답으로 하면 된다.

$N > 5$ 인 경우, 인접한 두 카드에 적힌 수를 모두 바꾸지 않아도 되는 경우가 반드시 존재한다. 그래서 모든 인접한 카드 쌍에 대해 그 카드들에 적혀있는 수를 바꾸지 않는 경우에 대해 $N \leq 5$ 인 경우와 비슷하게 답을 계산해주면 된다.

$O(N)$ 개 이하의 경우에 대해 바꿔야 하는 카드의 개수를 세는 데 $O(N)$ 의 시간이 걸린다. 따라서, 전체 시간복잡도는 $O(N^2)$ 이다. 제한 시간 내에 충분히 해결할 수 있다.

부분문제 3

부분문제 2의 풀이에서 설명했듯이, 수들을 적절히 바꾼 후 최종상태에서 카드에 적혀있는 수들은 적절한 정수 a, d 에 대해 왼쪽부터 순서대로 $a, a + d, \dots, a + (N - 1)d$ 가 돼야 한다.

그리고, 원래 카드에 적혀 있던 수 중에 적어도 하나는 바꾸지 않아도 된다는 점을 이용한다. i 번째 ($1 \leq i \leq N$) 카드를 바꾸지 않고, d 값이 정해진다면, $a = x - (i - 1)d$ 로 같이 정해지므로, 몇 개의 카드에 적혀있는 수들을 바꿔야 하는지 계산할 수 있다. 따라서 왼쪽부터 모든 카드를 순서대로 순회하면서, 가능한 모든 d 값에 대해 적혀있는 수를 바꿔야 하는 카드의 개수 중 최솟값을 계산해 주면 된다.

각 카드에 대해서, 시도 가능한 d 값의 개수가 $O(d)$ 이고, 각 경우에 대해서 적혀있는 수를 바꿔야 하는 카드의 개수를 세는 데 $O(N)$ 의 시간이 걸린다. 따라서 전체 시간복잡도는 $O(N^2d)$ 가 된다. d 의 범위가 작으므로, 제한 시간 내에 해결할 수 있다.

부분문제 4

먼저 답이 아무리 커도 $N - 2$ 이하임을 알 수 있다. 이는 왼쪽에서부터 첫 번째, 두 번째 카드에 적힌 수를 각각 a, b 라고 할 때, $d = b - a$ 로 두면 첫 번째, 두 번째 카드에 적힌 수는 바꿀 필요가 없다. 따라서 이 경우에 적혀있는 수를 바꿔야 하는 카드의 개수가 $N - 2$ 임을 알 수 있다. 그러므로 $N - 2$ 개 이하의 카드만 바꿔도 원하는 형태로 만들 수 있다.

답이 $N - 2$ 이하라는 것은, 적혀있는 수를 바꾸지 않아도 되는 카드가 적어도 2개는 있다는 것이다. 그러므로 모든 카드 쌍에 대해, 현재 보고 있는 카드가 왼쪽에서 i, j 번째 카드이고, ($i < j$) 적혀있는 수가 순서대로 x_i, x_j 라고 하자.

i, j 번째 카드를 바꾸지 않아도 되는 경우에 대해서 생각하면, 이때 $d = \frac{x_j - x_i}{j - i}$ 로 정해진다. d 가 정해졌으므로, $a = x_i - (i - 1)d$ 로 정해진다. a, d 가 정해졌으므로, 이 경우에 바뀌어야 할 카드의 개수를 정할 수 있다. 단, d 가 정수여야 하므로 $x_j - x_i$ 가 $j - i$ 의 배수여야 한다. 그렇지 않으면 불가능하다. 이 경우에는 넘어간다. 모든 경우에 대해 적혀있는 수를 바꿔야 하는 카드의 개수 중 최솟값을 답으로 하면 된다.

가능한 카드 쌍의 개수가 $\frac{N(N-1)}{2}$ 개이고, 각 경우에 대해 적혀있는 수를 바꿔야 하는 카드의 개수를 세는데 $O(N)$ 의 시간이 걸리므로, 전체 시간복잡도는 $O(N^3)$ 이 된다. 제한 시간 내에 전체 문제를 해결할 수 있다.

“트리와 쿼리” 문제 풀이

작성자: 윤교준

부분문제 1

가능한 트리의 형태가 세 가지밖에 없으므로, 조건문으로 쉽게 해결할 수 있다.

부분문제 2

S 에 속한 모든 두 정점쌍에 대하여 S 위에서 연결되어 있는지 여부를 판별하자.

두 정점이 S 위에서 연결되어 있는지는 DFS 등을 이용하면 $\mathcal{O}(N)$ 에 판별할 수 있다.

따라서, 각 쿼리당 $\mathcal{O}(NK^2)$ 에 문제를 해결할 수 있다.

부분문제 3

트리에서 S 에 속한 정점만 남기고 다른 모든 정점을 제거하면, 여러 개의 트리로 이루어진 포레스트(Forest)가 된다.

이 포레스트의 연결 컴포넌트를 “ S 위에서 연결 컴포넌트”라고 하자.

S 의 연결 강도는 S 위에서 연결 컴포넌트 각각의 크기를 통하여 계산할 수 있다.

따라서, S 에 속하는 정점만을 사용하여 DFS를 수행하면, 각 쿼리당 $\mathcal{O}(N)$ 에 문제를 해결할 수 있다.

일반적으로, 정점 집합의 부분 집합 $V' \subseteq V$ 를 방문하는 DFS의 시간 복잡도는 다음과 같다:

$$\sum_{v \in V'} \deg(v)$$

V' 에 속하는 각 정점 v 마다 $\deg(v)$ 개의 간선을 참조하기 때문이다.

이 문제의 경우, 일반적인 DFS를 수행한다면, 시간 복잡도는 다음과 같다:

$$\sum_{v \in S} \deg(v)$$

이 값은 집합의 크기 $|S| = K$ 보다 유의미하게 클 수 있음에 유의하라.

예를 들어, 1번 정점에 다른 모든 정점이 붙어 있는 트리 위에서 $S = \{1, 2, \dots, K\}$ 를 생각하자. $\deg(1) = N - 1$ 이므로 K 의 값에 무관하게, degree의 합이 $\mathcal{O}(N)$ 임을 확인할 수 있다.

부분문제 4

Union-find 기법을 응용하자.

입력에서 주어진 트리에서 아무 정점이나 잡아 루트로 만들자.

S 에 속하는 각 정점 v 마다, v 의 부모 정점이 S 에 속하는 경우에만 v 와 v 의 부모 정점을 union하자.

Union된 컴포넌트의 각 크기를 통하여 S 의 연결 강도를 계산할 수 있다.

부분문제 3과 다르게, S 에 속하는 각 정점 v 에 대하여 $\deg(v)$ 개의 간선이 아닌, 단 하나의 간선만을 참조함에 유의하라.

따라서, 각 쿼리당 $\mathcal{O}(K\alpha(N))$ 에 문제를 해결할 수 있다.

Union-find를 사용하지 않고 각 쿼리마다 정점을 깊이 순으로 정렬한 후, 깊은 정점부터 컴포넌트의 크기를 계산해 나가면서 $\mathcal{O}(N + \sum_i K_i \log K_i)$ 에 문제를 해결할 수도 있다. 쿼리를 오프라인으로 받을 경우, 계수 정렬을 사용할 수 있어서 위 알고리즘의 복잡도가 $\mathcal{O}(N + \sum_i K_i)$ 로 최적화된다.

“주차타워” 문제 풀이

작성자: 김준원

부분문제 1

모든 차의 번호가 1이기 때문에, 차를 어떤 순서로 빼도 괜찮다. 버튼 A를 $N - 1$ 번 누르거나, 버튼 B를 $N - 1$ 번 누르면 모든 차를 뺄 수 있다. 이 방법이 가장 적은 횟수로 차를 빼는 방법이다.

따라서, $N - 1$ 을 출력하면 된다.

부분문제 2

모든 차의 번호가 다르기 때문에, 차를 빼야 하는 순서가 유일하게 정해진다. 번호가 가장 작은 차부터 순서대로 빼어야 한다. 정해진 순서에 따라 이를 시뮬레이션하면 된다.

현재 맨 아래에 x 번째 칸이 있고, y 번째 칸에 있는 차를 빼야 하는 상황이라고 하자.

- $x \rightarrow y$ 방향으로 이동한다면 버튼을 $|x - y|$ 번 눌러야 한다.
- $x \rightarrow N \rightarrow 1 \rightarrow y$ 또는 $x \rightarrow 1 \rightarrow N \rightarrow y$ 방향으로 이동한다면 버튼을 $N - |x - y|$ 번 눌러야 한다.

따라서 차를 빼기 위해 버튼을 눌러야 하는 최소 횟수는 $\min(|x - y|, N - |x - y|)$ 번이다.

이 횟수를 N 대의 차를 빼는 순서대로 계산해서 합치면 문제의 답이 된다.

부분문제 3

N 개의 칸을 차례대로 모두 방문하는 순서는 최대 $N! = N \times (N - 1) \times \dots \times 2 \times 1$ 가지 존재한다.

$N!$ 가지의 순서를 백트래킹으로 모두 돌아보며, 어떤 순서로 방문하는 것이 가능한지, 가능하다면 이 때 버튼을 몇 번 누르게 되는지를 모두 계산하여 최적의 순서를 찾는다. 시간 복잡도는 $O(N \times N!)$ 이다.

부분문제 4

이제, 동적 계획법 (DP) 를 사용해야 이후의 부분문제들을 해결할 수 있다. 어떤 경로를 따라 차를 빼기 위해 버튼을 눌러야 하는 횟수를, 경로의 길이로 정의하자.

먼저, 지금은 차들의 번호의 대소관계만이 중요하므로, 좌표 압축 기법을 이용하여 번호를 1에서 N 사이에 있는 수들로 바꾸자. 그리고 번호가 1번인 차들의 목록, 번호가 2번인 차들의 목록, ...를 미리 저장해 두자.

$dp[X][E]$ 를, 번호가 1번에서 X 번까지인 모든 차들을 빼고 나서, 맨 아래에 E 번째 칸이 있도록 하기 위해 버튼을 눌러야 하는 최소 횟수로 정의하자. 그러면,

$$dp[X][E] = \min_{S=1}^N (dp[X-1][S] + \text{비용})$$

(비용: 현재 맨 아래에 S 번째 칸이 있을 때, 번호가 X 번인 모든 차를 빼고, 맨 아래에 E 번째 칸이 놓이게 하는 경로의 길이)

로 쓸 수 있다. 번호가 X 인 모든 차를 빼기 위해서는, 부분문제 1에서와 같이, 버튼 A를 계속 누르거나 버튼 B를 계속 눌러야 한다. 비용은 (1) S 번째 칸에서 출발해서 번호가 X 인 모든 차를 빼는 경로의 길이와, (2) 도착 지점에서 E 번째 칸으로 이동하는 경로의 길이의 합과 같다.

비용을 $O(1)$ 에 계산할 수 있으므로, 총 $O(N^3)$ 의 시간 복잡도에 부분문제 4를 해결할 수 있다.

부분문제 5

부분문제 4의 풀이에서, dp 배열의 모든 칸을 채울 필요는 없다. $dp[X][E]$ 의 값은, E 번째 칸에 있는 차의 번호가 X 번일 때에만 필요하다. 전체 dp 배열에서 채워야 하는 칸의 개수는 $O(N)$ 개이다. 따라서, $O(N^2)$ 의 시간 복잡도에 부분문제 5를 해결할 수 있다.

부분문제 6

풀이 1

문제를 푸는 데 필요한 실행 시간을 줄이려면, $dp[X][E]$ 한 칸을 계산하는 데 드는 시간을 $O(N)$ 미만으로 줄여야 한다. 풀이 1에서는 슬라이딩 윈도우 기법을 이용한다.

번호가 X 인 차가 총 K 개 있고, 각각 $a[0], \dots, a[K-1]$ 번째 칸에 있다고 하자. 출발 지점과 도착 지점에 상관 없이, K 대의 차들을 모두 빼는 방법은 아래 네 가지가 있다.

1. $a[0] \rightarrow a[K-1]$.
2. $a[K-1] \rightarrow a[0]$.
3. 어떤 인덱스 $0 \leq i < K-1$ 에 대해, $a[i] \rightarrow 1 \rightarrow N \rightarrow a[i+1]$.
4. 어떤 인덱스 $0 \leq i < K-1$ 에 대해, $a[i+1] \rightarrow N \rightarrow 1 \rightarrow a[i]$.

각각의 방법은 서로 다른 출발 지점과 도착 지점을 가진다. 앞서 부분문제 4에서 비용을 (1), (2) 두 가지로 분류했다. 각 방법에 대해, 우리는 $S =$ 해당 방법에서의 출발 지점인 경우만 고려하면 됨을 알고 있으므로, (1)을 $dp[X-1][\text{출발 지점}] +$ 해당 방법으로 이동하는 경로의 길이로 계산할 수 있다.

$dp[X][E]$ 는, 모든 도착 지점에 대해,

(1) 출발 지점에서 위 4가지 중 하나의 방법으로 도착 지점으로 도달하는 경로의 길이

(2) 도착 지점에서 E 로 이동하는 경로의 길이

를 합한 값의 최솟값으로 나타난다.

각 도착 지점에 대해, (1)은 dp 배열을 이용한 간단한 수식으로 계산할 수 있다. 이 값들을 모으고, 슬라이딩 윈도우 기법을 이용해, 각 E 에 대해 (1) + (2)의 최솟값을 구할 수 있다.

각 번호에 대해 $O(K)$ (K : 해당 번호를 가진 차의 수)의 시간 복잡도에 문제를 풀 수 있다. 따라서, 총 시간 복잡도는 $O(N)$ 이다.

풀이 2

앞서 풀이 1에서 $dp[X][:]$ 들을 한꺼번에 계산한 것과 다르게, $dp[X][S]$ 들을 일일이 계산할 것이다. 풀이 2에서는 게으르게 갱신하는 세그먼트 트리를 이용한다.

풀이 1에서, 같은 번호를 가진 차들을 모두 빼는 방법을 네 가지로 분류했었다. 이 차들을 빼기 위해 S 번째 칸에서 출발한다고 하자. 각각의 방법은, S 에서 출발해서 처음 빼는 차로 향해 이동할 때, 시계 방향으로

이동하는가 또는 시계 반대방향으로 이동하는가에 따라 두 가지씩으로 나누어진다. 각 방법에 대해 버튼을 눌러야 하는 최소 횟수는 시작하는 칸의 번호 S 에 대해 (1) $S + \text{상수}$ 또는 (2) $-S + \text{상수}$ 형태로 나타난다.

배열 $\text{arr}[1..N]$ 에 대해 다음 쿼리들을 지원하는 세그먼트 트리를 두 개 구성한다. 두 개의 세그먼트 트리는 각각 $S + \text{상수}$ 와 $-S + \text{상수}$ 로 나타내어지는 방법의 최소 이동횟수를 기록하기 위해 사용할 것이다.

1. $\text{init}()$: 트리를 초기화한다. (모든 값을 ∞ 로 교체한다)
2. $\text{update}(L, R, V)$: 각각의 $i = L, L+1, \dots, R-1, R$ 에 대해, $\text{arr}[i] = \min(\text{arr}[i], V)$ 연산을 시행한다.
3. $\text{query}(X)$: 배열의 X 번째 값을 구한다.

이제, 번호 1번을 가진 차들, 번호 2번을 가진 차들, $\dots$ 를 차례대로 순회하면서, 다음 번호에 해당하는 dp 배열의 값들을 채우기 위한 정보들을 두 개의 세그먼트 트리에 구성해 나간다.

게으르게 갱신하는 세그먼트 트리의 시간 복잡도가 $O(\log N)$ 이기 때문에, 총 시간 복잡도 $O(N \log N)$ 에 문제를 풀 수 있다.

첨부된 공식 풀이 소스코드는 풀이 2에 대한 소스코드이다.

“레벨 업” 문제 풀이

작성자: 이민제

부분문제 1

캐릭터를 레벨이 낮은 순서대로 정렬하고, 가장 레벨이 낮은 K 명의 캐릭터의 레벨을 올리는 방식으로 레벨 업 과정을 직접 시뮬레이션할 수 있다. 시간 복잡도는 $O(MN \log N)$ 이다.

부분문제 2

캐릭터를 레벨이 낮은 순서대로 정렬하자. $K = 1$ 인 경우에는 트레이닝이 다음과 같이 이루어짐을 알 수 있다.

- 첫 번째 캐릭터의 레벨이 계속해서 오른다.
- 첫 번째와 두 번째 캐릭터의 레벨이 같아지면, 이후 이 두 캐릭터의 레벨이 번갈아 가며 오른다.
- 첫 번째, 두 번째, 세 번째 캐릭터의 레벨이 같아지면, 이후 이 세 캐릭터의 레벨이 번갈아 가며 오른다.
- 이러한 과정은 모든 캐릭터의 레벨이 같아질 때까지 계속되며, 이후에는 N 개의 캐릭터의 레벨이 번갈아 가며 오른다.

각 $1 \leq i \leq N$ 에 대해, 첫 i 개 캐릭터의 레벨이 같아지는 시점을 누적 합 등을 이용하여 계산할 수 있다. 이 시점이 M 을 초과하지 않으면서 최대가 되도록 하는 i 를 k 라 하자. 이때 $(k+1)$ 번째, $\dots$, N 번째 캐릭터는 레벨이 오르지 않음을 알 수 있다. 한편 1번째 캐릭터와 k 번째 캐릭터의 레벨 차이는 최대 1이므로, 전체 캐릭터의 레벨 총합을 이용하여 각각의 레벨을 구할 수 있다.

시간 복잡도는 $O(N \log N)$ 이다.

부분문제 3

부분문제 3에서는 M 이 크지 않으므로, 각각의 트레이닝을 빠르게 시뮬레이션하는 방식으로 문제를 해결할 수 있다.

캐릭터가 레벨이 낮은 순서대로 정렬되어 있다고 가정하고, K 번째 캐릭터의 레벨을 x 라고 하자. 또, 레벨이 x 미만인 캐릭터의 수를 l , x 이하인 캐릭터의 수를 r 라고 하자. 트레이닝을 한 번 진행하면 캐릭터의 레벨은 다음과 같이 변한다.

- 1번째, $\dots$, l 번째 캐릭터의 레벨이 1 오른다.
- $(l+1)$ 번째, $\dots$, r 번째 캐릭터 중 $(K-l)$ 명의 캐릭터의 레벨이 1 오른다. 편의상 $(l+r-K+1)$ 번째, $\dots$, r 번째 캐릭터의 레벨이 오른다고 하자. 그러면 트레이닝 이후에도 캐릭터들의 레벨이 정렬된 상태를 유지하게 된다.

따라서 정렬된 배열에서 다음과 같은 작업을 효율적으로 수행할 수 있으면 된다.

- 위에서 정의한 x , l , r 을 구한다.

- 구간 $[1, l]$, $[(l + r - K + 1), r]$ 에 1을 더한다.

이는 Lazy Propagation을 사용한 Segment Tree로 빠르게 처리할 수 있다. 시간 복잡도는 $O(M \log N)$ 이다.

부분문제 4

부분문제 2와 마찬가지로, 트레이닝이 진행됨에 따라 캐릭터들의 레벨이 점점 일정해진다는 사실을 관찰할 수 있다. 단, 부분문제 2에서는 레벨이 가장 낮은 캐릭터를 중심으로 레벨이 일정해졌지만, 일반적인 경우에는 레벨이 K 번째로 낮은 캐릭터를 중심으로 레벨이 일정해진다.

맨 처음 상태에서 K 번째로 낮은 레벨을 x 라 할 때, N 명의 캐릭터를 다음과 같이 세 그룹으로 나누자.

- A그룹: 레벨이 x 미만.
- B그룹: 레벨이 x 이상 $x + 1$ 이하.
- C그룹: 레벨이 $x + 1$ 초과.

트레이닝을 진행하면 각 그룹에서 캐릭터들의 레벨은 다음과 같이 변한다.

- A그룹: 모든 캐릭터의 레벨이 1 오른다.
- B그룹: 레벨이 낮은 $(K - |A|)$ 명의 레벨이 1 오른다. (단, $|A|$ 는 그룹 A의 크기.)
- C그룹: 레벨이 오르지 않는다.

트레이닝을 진행하다 보면 A그룹 혹은 C그룹의 캐릭터가 B그룹의 레벨 범위 안으로 들어오는 경우가 발생하는데, 이 경우 해당 캐릭터를 B그룹으로 옮기자. 이때 다음의 사실들을 관찰할 수 있다.

- 레벨이 K 번째로 낮은 캐릭터는 항상 B그룹에 속한다. 즉, 위에서 서술한 각 그룹의 성질은 계속해서 유지된다.
- B그룹 내에서 캐릭터 간의 레벨 차이는 1 이하이다. 따라서 B그룹의 크기와 레벨 범위, 각 레벨에 해당하는 캐릭터의 수를 알면 B그룹 전체의 정보를 얻게 된다. 이 값들을 관리하자.
- A그룹의 캐릭터는 레벨이 높은 순서대로 B그룹으로 들어오며, C그룹의 캐릭터는 레벨이 낮은 순서대로 B그룹으로 들어온다.

A그룹에서 레벨이 가장 높은 캐릭터 또는 C그룹에서 레벨이 가장 낮은 캐릭터가 B그룹으로 들어오는 과정을 시뮬레이션하자. 이 캐릭터가 B그룹으로 들어오는 시점은 앞서 관리한 B그룹의 정보를 바탕으로 이분 탐색(Parametric Search)을 이용하여 계산할 수 있다.

이를 반복하여 각 캐릭터가 B그룹으로 들어오는 시점을 모두 구하자. B 그룹으로 새로운 캐릭터가 들어오는 경우가 총 $O(N)$ 번 발생하므로, 위 과정을 $O(N \log M)$ 시간에 모두 시뮬레이션할 수 있다.

트레이닝이 종료된 시점에서 각 캐릭터가 어느 그룹에 속하는지 안다면, 각각의 레벨 또한 어렵지 않게 구할 수 있다. 총 시간복잡도는 $O(N(\log N + \log M))$ 이다.